DÉFI TECHNIQUE TETON · PLATEFORME · RAPPORT DE SOLUTION

Provisionner le Wi-Fi d'un appareil sans écran. Par Bluetooth, sans réseau partagé.

Un technicien transmet les identifiants Wi-Fi à un appareil Ubuntu sans écran, depuis son téléphone et par Bluetooth Low Energy, sans Internet ni réseau commun. L'appareil rejoint ensuite le réseau du site. Ce rapport décrit la solution retenue, ce qui est construit et prouvé, et ce qui reste à faire.

Auteur	Noé Breton
Date	18 octobre 2026
Dépôt	`teton-provisioning`, base `a9551fa` et travail en cours
Appareil de test	PC portable Ubuntu 22.04.5, BlueZ 5.64, NetworkManager 1.36.6, puce MediaTek MT7921

46
tests Rust verts, debug et release

91
contrôles Playwright de l'interface, 0 échec

4/4
contrôles réussis sur l'hôte réel, annonce BLE active

0
échange radio avec un téléphone à ce jour

Construit et vérifié

- Service appareil Rust sur BlueZ : sonde `hello` → `world` en GATT, annonce réelle vérifiée sur l'hôte.
- Simulation, benchmarks et audit de l'hôte en lecture seule.
- Maquette de l'interface : réseau du site saisi une fois, puis chambre par chambre ; testée sur PC et mobile.
- Revue de sécurité du transfert des identifiants, avec revue indépendante.

Pas encore fait

- Échange avec un vrai téléphone Android et iPhone.
- Canal chiffré, transfert des identifiants, connexion Wi-Fi réelle.
- Application mobile du technicien.

Le défi n'est pas encore complet. Aucune preuve ne montre des identifiants transmis ni une connexion Wi-Fi réelle. Les résultats simulés sont étiquetés comme tels et ne valident aucune étape matérielle.

1 Les quatre contraintes et nos réponses
2 Choix du canal local
3 Architecture
4 Sonde hello sur GATT
5 Protection des identifiants en transit
6 Interface du technicien et tests Playwright
7 200 appareils dans une aile d'hôpital
8 Preuves, reproduction et suite

Les quatre contraintes. Chacune a sa réponse et son état.

Le défi impose quatre contraintes. Pour chacune, le tableau donne la réponse retenue et l'état réel des preuves au 18 octobre 2026. « Code prêt » ne veut pas dire « prouvé sur la radio ».

Contrainte du défi	Réponse retenue	État
Pas d'Internet, pas de réseau local partagé au départ	BLE direct entre le téléphone et l'appareil. Application préparée avant l'intervention, ressources hors ligne. Aucun point d'accès temporaire.	● Annonce réelle OK, téléphone à tester
Connexion à un vrai réseau Wi-Fi	Le service active le profil via NetworkManager par D-Bus, puis renvoie l'état frais de cette connexion et son IP.	● Non commencé
Pas d'identifiants en clair dans l'air	Canal applicatif Noise `NKpsk2` : la clé publique de l'appareil et un secret sont imprimés sur son étiquette QR.	● Conçu et revu
Utilisable par un non-développeur	Réseau du site saisi une seule fois. Ensuite, chambre par chambre : scanner les étiquettes des capteurs, un bouton, un statut par capteur et des relances ciblées. Rien à taper sur l'appareil.	● Maquette testée

2 · CANAL

Pourquoi le Bluetooth Low Energy. Et ce qu'il coûte.

Canal	Sans réseau partagé	Android et iPhone	Non-développeur	Verdict
BLE GATT	Oui	Oui, avec une app préparée	Oui : scanner, toucher	Retenu
Point d'accès temporaire (SoftAP)	Ambigu : crée un réseau local	Oui, sans app	Oui	Écarté
QR affiché par l'appareil	Oui	Oui	Oui	Pas d'écran
NFC	Oui	Selon le téléphone	Oui	Pas de lecteur
Câble USB	Oui	Adaptateurs variables	Non	Écarté

Ce que le BLE apporte

- Présent sur tous les téléphones et sur la plupart des appareils Linux.
- Aucun réseau à créer : on lit la contrainte du défi au plus strict. Le SoftAP aurait évité l'app, mais il crée un réseau local (décision 18).
- Portée de quelques mètres, adaptée à un geste dans une chambre.

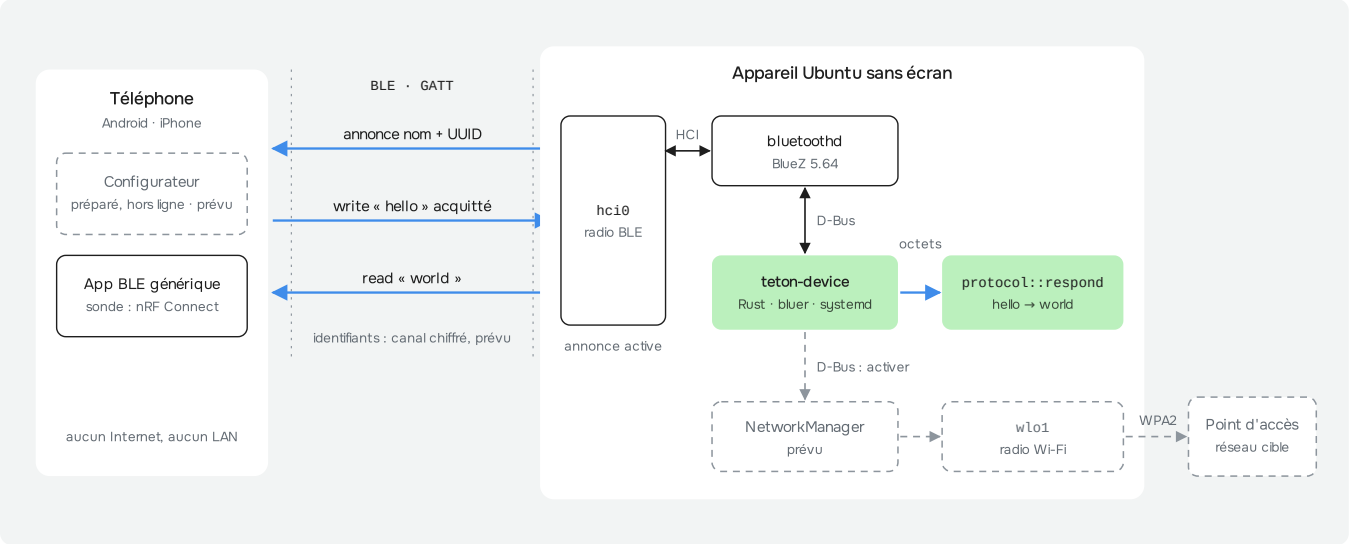
Ce qu'il coûte

- Une application à préparer et installer sur chaque plateforme.
- Le pairing natif n'authentifie pas un appareil sans écran : il faut un chiffrement applicatif.
- Petits paquets (MTU) : il faut découper les messages chiffrés.

3 · ARCHITECTURE

Topologie. Deux radios, un seul service.

Le service Rust `teton-device` parle à BlueZ, et plus tard à NetworkManager, uniquement par D-Bus. Aucun mot de passe ne passe en argument de processus. Le gestionnaire partagé `protocol::respond` ne connaît ni le Bluetooth ni le Wi-Fi : il reçoit des octets et renvoie des octets. systemd démarre le service au boot, sans écran ni SSH.



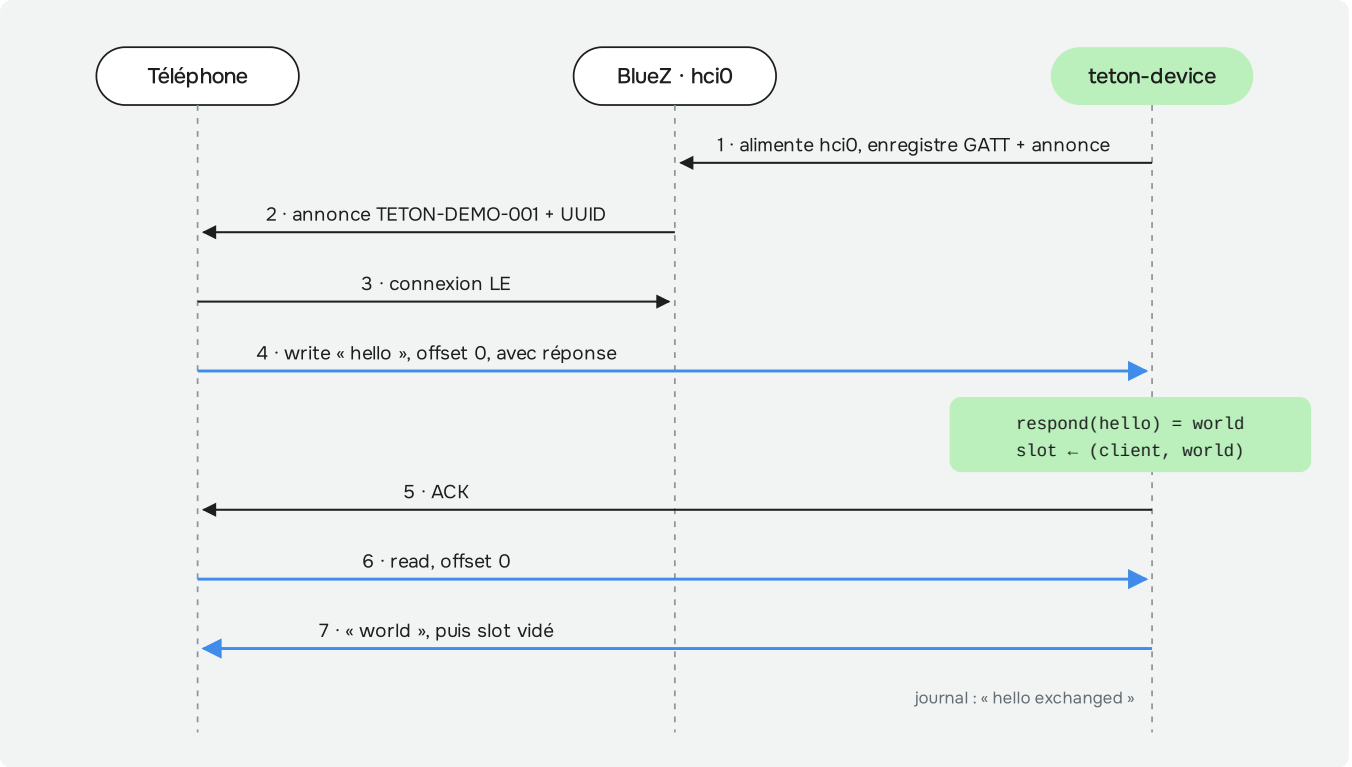
En vert menthe, le code de ce dépôt ; en bleu, l'échange BLE de la sonde ; en pointillé, ce qui est prévu mais pas construit. Les radios BLE et Wi-Fi partagent la même puce MT7921 sur l'hôte de test.

Fichier	Rôle	État
`src/protocol.rs`	Contrat partagé : hello → world, UUID du service et de la caractéristique	Testé
`device/main.rs`	Périphérique BLE sur BlueZ via `bluer` 0.17.4, feature optionnelle `ble`	6 tests unitaires, radio non testée
`device/systemd/teton-device.service`	Démarrage au boot, utilisateur dynamique, suit BlueZ	Écrit, non installé
`device/audit.rs`	Pré-vol de l'hôte en lecture seule, délais bornés	Hôte réel : 4/4
`src/simulation.rs`, `batch.rs`	Transport simulé, pannes injectées, lots bornés	Testé, mesuré
`docs/architecture-demo.html`	Maquette de l'interface technicien, hors ligne	91 contrôles Playwright
`configurator/`	Application Android et iPhone	Absente : framework non choisi

4 · PREMIER JALON

Sonde hello sur GATT. Prouver la radio avant les secrets.

Avant tout identifiant, l'appareil expose un service GATT inoffensif : une caractéristique en écriture acquittée et en lecture. Il garde une seule réponse en attente, liée au client qui a écrit. Toute écriture vide d'abord ce slot ; une lecture rend `world` une fois, et seulement à ce client.



Écriture invalide : refus `NotSupported`. Offset non nul : refus `InvalidOffset`. Le journal ne contient jamais le contenu écrit ni l'adresse du téléphone.

Tests

6 tests unitaires sur la logique du slot. 4 mutations du code (contrôle du client, vidage, offsets) font chacune échouer un test.

Revue indépendante

Approuvée après un correctif : trois mutations passaient les premiers tests. Taille d'annonce et droits D-Bus vérifiés.

Exécution réelle

Sur l'hôte, BlueZ confirme l'adaptateur allumé, une annonce active et le service publié ; tout est libéré à l'arrêt. Aucun téléphone ne l'a encore vu.

Protéger les identifiants. Sans écran sur l'appareil.

L'appareil n'a ni écran ni clavier, et le même parcours doit marcher sur Android et sur iPhone. Le pairing Bluetooth natif ne peut donc pas authentifier l'appareil de façon fiable. Nous recommandons un canal chiffré au niveau de l'application. Cette partie est conçue et revue, mais pas construite.

Option	Raison	Verdict
Pairing BLE « Just Works »	Chiffre le lien mais n'authentifie personne : un intermédiaire actif gagne.	Rejeté
Code fixe imprimé (passkey)	Un attaquant actif le retrouve en une vingtaine de tentatives (CVE-2020-26558).	Rejeté
Pairing OOB avec la clé du QR	Indisponible pour les apps iOS ; API système hors SDK public sur Android.	Rejeté
Noise `NKpsk2`, clé et secret dans le QR	Fonctionne en GATT simple sur les deux plateformes. L'appareil prouve sa clé, le téléphone prouve qu'il a lu l'étiquette. Aucune boîte de dialogue système.	Recommandé
SRP6a avec secret d'étiquette (modèle ESP-IDF)	Précédent éprouvé en provisioning BLE ; pile SRP moins courante côté appareil.	Alternative

Flux prévu

- **Atelier.** Une commande locale crée la clé et le secret, puis imprime l'étiquette QR. Jamais par BLE, jamais régénérés au démarrage.
- **Poignée de main.** `Noise_NKpsk2_25519_ChaChaPoly_SHA256`, avec la bibliothèque Rust `snow` 0.10. Rien de secret avant la fin de l'échange.
- **Envoi.** SSID, mot de passe et identifiant de tentative, chiffrés et intègres (AEAD).
- **Activation.** NetworkManager par D-Bus ; le résultat revient dans le canal, lié à la tentative et à l'IP observée.

Règles issues de la revue de sécurité

- **Usage unique.** Après un succès, l'appareil ferme le service et ne le rouvre jamais au redémarrage. Il faut alors un jeton signé par Smith ou un passage en atelier.
- **Une tentative à la fois**, état par connexion ; un second téléphone est refusé.
- **Rejeu.** Clés éphémères fraîches ; identifiant de tentative lié au contenu envoyé.
- **Journaux.** Résultats et identifiants de tentative seulement.

Risque accepté pour la démo. Une étiquette QR falsifiée, collée sur la vraie, n'est pas détectée. Pour un déploiement réel, le téléphone vérifiera la clé dans une liste d'appareils signée par Smith. Détails et tests obligatoires : `docs/SECURITY.md`.

Le parcours du technicien. Le réseau une fois, puis chambre par chambre.

Une chambre compte souvent plusieurs capteurs. Le technicien saisit le réseau du site une seule fois, puis, dans chaque chambre, scanne les étiquettes et connecte tous les capteurs d'un geste. La maquette est aux couleurs de teton.ai et ses actions sont simulées. Captures Playwright en émulation iPhone 13.



1 · Réseau du site, une fois



2 · Scanner la chambre



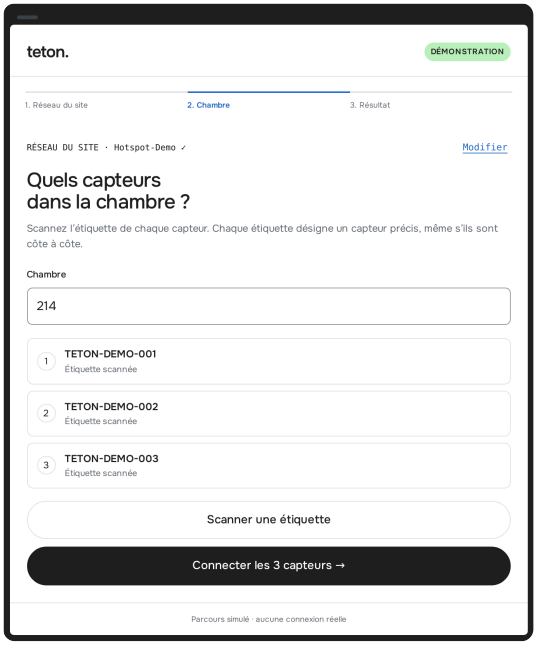
3 · Un capteur à la fois



4 · Chambre prête



5 · Relance ciblée



Aperçu ordinateur. Tests sur PC 1440 × 900, iPhone 13 et Pixel 7 émulés dans Chromium ; le mot de passe quitte le formulaire dès l'enregistrement du site. Pas de vrai Safari ni de téléphone physique.

Contrôle Playwright	Résultat
5 scénarios × 2 aperçus × 3 formats, avec correction	● 54 réussis
Chambre suivante sans ressaisir le mot de passe	● 6 réussis
Pas de défilement horizontal, police chargée, aucune erreur console ni CSP	● 15 réussis
Cibles tactiles d'au moins 44 px	● 6 réussis
Accessibilité axe-core, WCAG 2.1 AA, 3 écrans × 3 formats	● 0 violation
Parcours complet au clavier, focus sur chaque titre d'écran	● Réussi

200 appareils dans une aile d'hôpital. Proposition, non mesurée.

Les mesures de simulation (lots de 1 000 tâches) valident la logique du logiciel, pas la capacité radio. Voici ce qui changerait pour une équipe d'installation non technique.

Pour l'équipe terrain

- Le réseau du site est saisi une fois par intervention, ou chargé depuis un profil signé par Smith.
- Chambre par chambre : numéro de chambre, scan des étiquettes, un bouton. L'app note quel capteur est dans quelle chambre.
- Plusieurs opérateurs travaillent en parallèle, chacun sur un étage, avec une liste de lot à cocher.
- Préparation en atelier quand c'est possible : identité, étiquette, image système.
- Les exceptions remontent dans une liste : prêts, échecs avec la cause, inconnus.

Pour le système

- File d'attente bornée dans l'app, quelques connexions BLE simultanées par téléphone. La limite reste à mesurer.
- Annonce coupée après succès : moins de bruit radio, surface d'attaque réduite.
- Liste d'appareils signée par Smith et préchargée : une étiquette falsifiée est détectée.
- Après connexion, l'appareil s'enregistre auprès de Smith, qui suit l'avancement du lot.

Identifiants : qui partage quoi ?

- **Secret d'étiquette :** unique à chaque capteur ; il ne sert qu'à ouvrir le canal BLE chiffré.
- **Réseau du site :** en WPA2-Personal, tous les capteurs reçoivent le même mot de passe. Un capteur volé expose alors le réseau, et révoquer un capteur oblige à changer le mot de passe partout.
- **En production :** des identifiants propres à chaque appareil, émis par Smith et liés à l'identifiant scanné : certificat EAP-TLS, ou clé par appareil (PPSK) sur le même réseau. Le geste du technicien ne change pas.

Radio

200 annonces dans un couloir : filtrer par UUID et par identifiant du QR, et ne jamais choisir par la seule force du signal.

Reprise

Chaque tentative a un identifiant. Après une coupure, l'app interroge l'appareil au lieu de renvoyer le mot de passe.

Sécurité

Provisioning à usage unique ; un reprovisioning demande un jeton signé. Aucune fenêtre ne se rouvre au redémarrage.

Ce qui est prouvé. Chaque classe de preuve est suivie à part.

Classe	État	Contenu	Fichier
SIMULATION	● Réussi	Transport simulé, pannes, soak de 300 s ; maquette testée par Playwright	`demo/benchmark*.txt`, `demo/mockup-playwright.txt`
COMMAND_FIXTURE	● Réussi	46 tests Rust, mutations du service BLE, 4 cas CLI	`demo/tdd-ble-device.txt`
ACTUAL_HOST	● Réussi 4/4	Adaptateur allumé, annonce et service publiés puis libérés à l'arrêt	`demo/host-ble-endpoint.txt`, `demo/host-inventory.txt`
ACTUAL_BLE	● Non exécuté	Échange avec Android et iPhone	—
Canal sécurisé	● Non exécuté	Conçu et revu, pas codé	`docs/SECURITY.md`
ACTUAL_WIFI	● Non exécuté	Connexion réelle au réseau cible	—

Reproduire sur Ubuntu

```
sudo apt install libdbus-1-dev pkg-config
rustup toolchain install 1.88.0 --profile minimal --component rustfmt --component clippy
cargo test --all-targets --features ble --locked
cargo build --release --features ble --bin device --locked
sudo rfkill unblock bluetooth        # si « Soft blocked: yes »
./target/release/audit-host && ./target/release/device
# Téléphone (nRF Connect) : se connecter à TETON-DEMO-001, écrire « hello », lire « world »
```

Prochaines étapes

- Sonde hello depuis Android et iPhone : découverte, échange, reconnexion.
- Installer le service systemd ; tester le redémarrage du service et le reboot.
- Choisir le framework mobile, puis construire le canal Noise et ses tests obligatoires.
- Activation NetworkManager, statut frais lié à la tentative, preuves Wi-Fi réelles.

Limites connues

- Si l'adaptateur disparaît sans redémarrage de BlueZ, le service tourne sans être visible.
- BLE et Wi-Fi partagent une puce sur l'hôte de test : coexistence à vérifier.
- Police Onest à la place de Wallop, qui est sous licence ; couleurs inspirées de teton.ai.
- Document de candidature, non officiel.